

AUTONOMOUS CATAMARAN – SOFTWARE ARCHITECTURE (ROS 2 HUMBLE)

SYSTEM LAYERS

1. Operating System & Base Layer

- Ubuntu 22.04 LTS (arm64) on Raspberry Pi 4B
- ROS 2 Humble installed
- `systemd` or `launchctl` (if needed) for node auto-start on boot

ROS 2 NODE ARCHITECTURE

Sensor Nodes

Node Name	Type	Topic Published	Purpose
<code>lidar_node</code>	C++ / Python	<code>/scan</code>	2D LIDAR data for mapping & nav
<code>ultrasonic_node</code>	Python	<code>/ultrasonic_range</code>	Short-range front obstacle check
<code>servo_ultrasonic_node</code>	Python	<code>/scan_servo_range</code>	Side-scan sonar-like sweep
<code>gps_node</code>	Python	<code>/fix</code> (<code>sensor_msgs/NavSatFix</code>)	GNSS-based global positioning
<code>imu_node</code>	C++	<code>/imu/data</code>	Orientation, heading, stability
<code>voltage_node</code>	Python	<code>/power/voltage</code>	Battery voltage via ADC
<code>current_node</code>	Python	<code>/power/current</code>	Battery current via ADC

Control Nodes

Node Name	Type	Topic Subscribed / Published	Purpose
<code>thruster_controller</code>	Python	<code>/cmd_vel</code> → PWM GPIO	Drive T100 ESCs (Left, Right)
<code>pid_controller</code>	Python	<code>/imu/data</code> , <code>/fix</code> → <code>/cmd_vel</code>	Velocity & heading correction
<code>waypoint_navigator</code>	Python	<code>/fix</code> , <code>/imu/data</code>	Navigation to GPS waypoints
<code>obstacle_avoidance</code>	Python	<code>/scan</code> , <code>/ultrasonic_range</code>	Emergency avoidance logic

Power Management Nodes

Node Name	Type	Topic Published	Purpose
<code>power_monitor</code>	Python	<code>/power/status</code>	Combines voltage & current
<code>solar_status_node</code>	Python	<code>/solar/input</code> (optional)	Track solar input if any

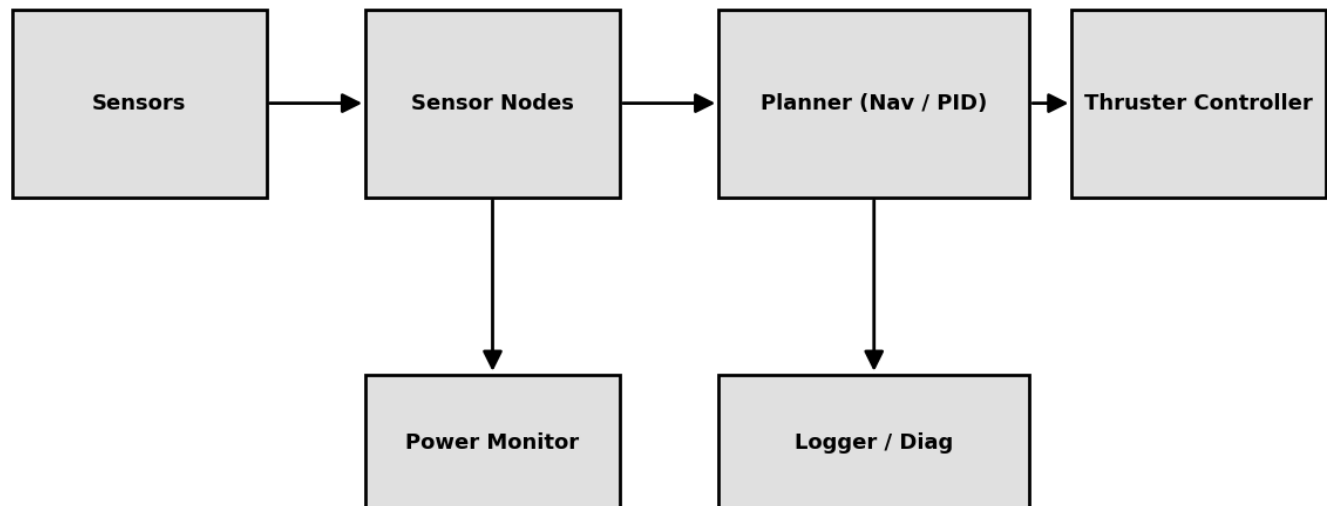
System/Debug Nodes

Node Name	Type	Purpose
logger_node	Python	Log power, position, and status
system_health_node	Python	Monitor heartbeat of all nodes
diagnostic_aggregator	Built-in	Aggregate diagnostic status

CORE ROS 2 TOPICS

/sensor/scan	← rplidar_node
/sensor/ultrasonic	← ultrasonic_node
/sensor/ultrasonic_servo	← servo_ultrasonic_node
/sensor/gps/fix	← gps_node
/sensor/imu/data	← imu_node
/sensor/power/voltage	← voltage_node
/sensor/power/current	← current_node
/cmd_vel	→ thruster_controller (twist messages)
/thruster/pwm	← converted cmd_vel to GPIO PWM
/power/status	← power_monitor
/navigation/goal	← waypoint_navigator
/diagnostics	← system_health_node, diagnostic_aggregator

DATA FLOW DIAGRAM



BUILD & DEPLOYMENT

- **Workspace:** catamaran_ws
- **Packages:**
 - **sensors_pkg:** Contains all sensor nodes
 - **thruster_pkg:** Contains thruster + PID control nodes
 - **nav_pkg:** Waypoint follower + GPS/IMU integration
 - **power_pkg:** Power monitoring scripts
 - **system_pkg:** Launch files, diagnostics, logger
- **Launch:** One `bringup.launch.py` to start all nodes

OPTIONAL ADD-ONS

- RViz2 / Foxglove for visualization
- rosbag2 for logging sensor data
- Docker container to test on PC before Pi deployment

SUGGESTED FOLDER STRUCTURE

```
catamaran_ws/  
└─ src/  
    └─ sensors_pkg/  
        ├── lidar_node.py  
        ├── gps_node.py  
        └─ ...  
    └─ thruster_pkg/  
        ├── thruster_controller.py  
        └── pid_controller.py  
    └─ nav_pkg/  
        └── waypoint_navigator.py  
    └─ power_pkg/  
        ├── voltage_node.py  
        └── current_node.py  
    └─ system_pkg/  
        ├── bringup.launch.py  
        └── system_health_node.py
```